CUDA KERNEL GEN CLIENT REPORT

Up to 80.6x drop-in speedup for the original CUDA benchmark

Coalesced vectorization and compact data layouts for a transcendental CUDA grid benchmark, while preserving the client's float input/output contract for the recommended default.

ORIGINAL BASELINE	RECOMMENDED DROP-IN	FASTEST KERNEL-ONLY
14.8900 ms	**80.6x**	**638.7x**
Strict problem definition	0.1848 ms	0.0233 ms

ADJACENT L2 PIPELINE	BEST SCORE PATH
325.0x	**595.9x**
0.0458 ms	0.0250 ms

Executive Summary

The baseline problem processes a fixed 8192 x 8192 float grid with five dependent transcendental iterations per element and a 1e-3 relative tolerance check.

The recommended production default is the vectorized float kernel. It keeps the input/output contract intact and moves the runtime from 14.8900 ms to 0.1848 ms on NVIDIA B300 SXM6 AC.

The fastest specialized kernel combines uint4-packed U8 input/output with persisting L2 on the compact input. The adjacent L2 pipeline is reported separately because it times useful downstream consumer work too; the fused score path is the best score-producing compact-data result in this report.

Nsight classifies the drop-in float path as compute-throughput sensitive on the measured GPU, which explains why fixed-range affine math can win materially here. The compact fused paths still matter because they remove global-memory traffic and launch boundaries.

Recommendation

Adopt the vectorized float kernel as the production default.

Use FP16 output or compact U8 paths only when the product can adopt those data contracts. They are compelling when surrounding stages can store or consume compact data directly.

Client Options

DROP-IN

Vectorized float default

Recommended when the float input/output contract must stay unchanged.

80.6x

0.1848 ms

512 MiB data moved

OUTPUT CONTRACT

FP16 output

Useful when the client can accept half-precision output storage.

236.7x

0.0629 ms

256 MiB data moved

COMPACT CONTRACT

Compact U8 input + FP16 output

Best when an upstream producer can emit only the two changing values directly.

374.1x

0.0398 ms

160 MiB data moved

SPECIALIZED CONTRACT

uint4 + L2 U8 in/out kernel

Fastest kernel-only result: compact U8 input/output with persisting input L2.

638.7x

0.0233 ms

64 MiB data moved

SPECIALIZED PIPELINE

uint4 + L2 compact pipeline

Best adjacent producer-consumer path: uint4 producer plus persisting compact output.

325.0x

0.0458 ms

160 MiB data moved

FUSED PIPELINE

Fused compact U8 score + L2

Fastest score-producing path if compact input is reused and can stay resident in L2.

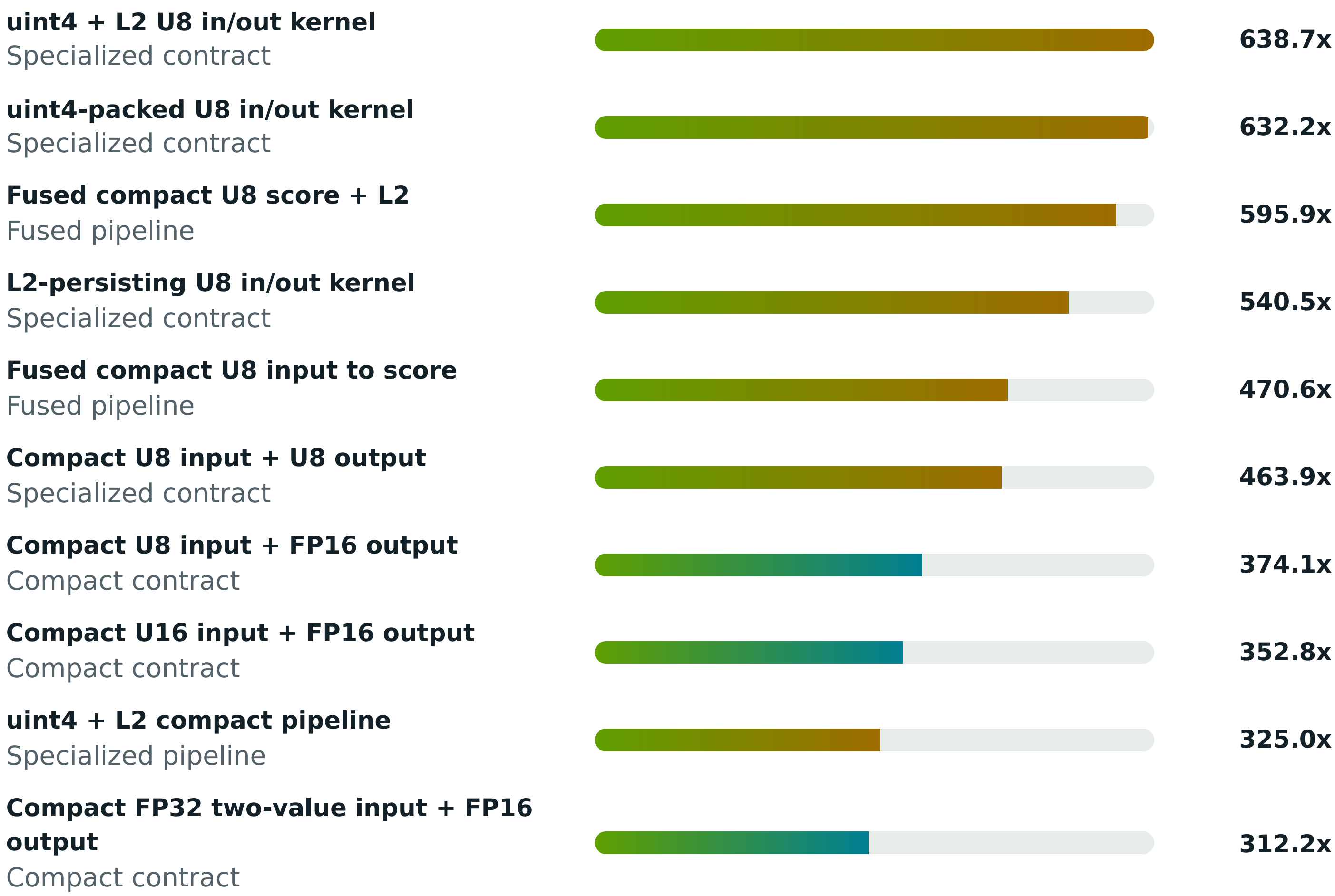
595.9x

0.0250 ms

96 MiB data moved

Performance Ladder

Client-relevant variants, sorted by speedup against the original client baseline. L2-resident results are included when they were measured in the current report.



Why The Speedup Holds

Data movement falls as the contract narrows

The drop-in path fixes memory access geometry. Data-contract-changing paths then reduce output bytes and keep only the two lanes that actually vary in this benchmark.

Compact U8 input + U8 output	**64 MiB**
Compact U8 input + FP16 output	**160 MiB**
FP16 output	**256 MiB**
Vectorized float default	**512 MiB**
Original row-stride kernel	**512 MiB**

Profiler evidence

The raw L1 sector count is a transaction counter: it confirms that vectorized loads and stores are coalesced into predictable memory chunks. The client takeaway is bandwidth pressure, not a need to inspect sector math.

MEMORY BANDWIDTH
34.29%
of sustained DRAM throughput

COMPUTE UTILIZATION
95.80%
arithmetic headroom remains

OCCUPANCY
92.61%
Healthy scheduling

MEMORY ACCESS
Coalesced
8.4M memory chunks

Ranked Benchmark Portfolio

Sorted by speedup vs the original client baseline, including L2-resident rows when available. Footprint combines data moved, register pressure, and spill status so the table stays decision-oriented.

VARIANT	TRACK	STATUS	SPEEDUP	TIME	FOOTPRINT
uint4 + L2 U8 in/out kernel Fastest kernel-only result: compact U8 input/output with persisting input L2.	SPECIALIZED CONTRACT	Pass	**638.7x**	**0.0233 ms**	64 MiB data moved
uint4-packed U8 in/out kernel Processes eight compact groups per thread; faster producer without relying on L2 hints.	SPECIALIZED CONTRACT	Pass	**632.2x**	**0.0236 ms**	64 MiB data moved
Fused compact U8 score + L2 Fastest score-producing path if compact input is reused and can stay resident in L2.	FUSED PIPELINE	Pass	**595.9x**	**0.0250 ms**	96 MiB data moved
L2-persisting U8 in/out kernel Fastest kernel-only result when compact input is reused and fits in persisting L2.	SPECIALIZED CONTRACT	Pass	**540.5x**	**0.0276 ms**	64 MiB data moved
Fused compact U8 input to score Reads compact U8 input and writes score output directly, skipping the compact output buffer.	FUSED PIPELINE	Pass	**470.6x**	**0.0316 ms**	96 MiB data moved
Compact U8 input + U8 output Fastest kernel-only result; does not include a downstream consumer.	SPECIALIZED CONTRACT	Pass	**463.9x**	**0.0321 ms**	64 MiB data moved 14 registers/thread 0 spill bytes
Compact U8 input + FP16 output Best when an upstream producer can emit only the two changing values directly.	COMPACT CONTRACT	Pass	**374.1x**	**0.0398 ms**	160 MiB data moved 28 registers/thread 0 spill bytes
Compact U16 input + FP16 output Preserves tolerance while storing only the two changing values in each four-value group.	COMPACT CONTRACT	Pass	**352.8x**	**0.0422 ms**	192 MiB data moved 30 registers/thread 0 spill bytes
uint4 + L2 compact pipeline Best adjacent producer-consumer path: uint4 producer plus persisting compact output.	SPECIALIZED PIPELINE	Pass	**325.0x**	**0.0458 ms**	160 MiB data moved
Compact FP32 two-value input + FP16 output Upper-bound compact input shape before fixed-point quantization.	COMPACT CONTRACT	Pass	**312.2x**	**0.0477 ms**	256 MiB data moved 30 registers/thread 0 spill bytes
uint4 U8 producer + consumer Vectorized compact producer followed by the existing compact consumer.	PIPELINE	Pass	**288.3x**	**0.0517 ms**	160 MiB data moved
L2-resident U8 in/out pipeline U8 input, U8 output, and compact consumer timed together with persisting L2.	SPECIALIZED PIPELINE	Pass	**238.2x**	**0.0625 ms**	160 MiB data moved
FP16 output Useful when the client can accept half-precision output storage.	OUTPUT CONTRACT	Pass	**236.7x**	**0.0629 ms**	256 MiB data moved 30 registers/thread 0 spill bytes

VARIANT	TRACK	STATUS	SPEEDUP	TIME	FOOTPRINT
Affine float default Documents the fixed-range math opportunity, but does not materially beat the float default.	DROP-IN-SPECIALIZED	Pass	**176.0x**	**0.0846 ms**	512 MiB data moved 22 registers/thread 0 spill bytes
Compact U8 output + score pipeline Setup-paid compact-output pipeline; retained as a comparison, not the extreme path.	PIPELINE	Pass	**128.1x**	**0.1162 ms**	448 MiB data moved
Vectorized float default Recommended when the float input/output contract must stay unchanged.	DROP-IN	Pass	**80.6x**	**0.1848 ms**	512 MiB data moved 26 registers/thread 0 spill bytes
Scalar coalesced kernel Shows the value of coalesced memory access before vectorization.	STRUCTURAL FIX	Pass	**20.7x**	**0.7197 ms**	512 MiB data moved 21 registers/thread 0 spill bytes
Original row-stride kernel Same harness, inefficient memory layout	KERNEL BASELINE	Pass	**2.3x**	**6.4807 ms**	512 MiB data moved 19 registers/thread 0 spill bytes

Maximum-Performance L2 Path

When a compact data contract is worth considering

This path is aimed at latency-sensitive customers who can own the compact data contract, producer/consumer coupling, and maintenance burden. It is not the safe library default.

There are two useful combinations. First, the current best kernel-only shape uses uint4 loads/stores to process eight compact groups per thread, then keeps compact U8 input in persisting L2; that measured 0.0233 ms, or 638.7x against the original client baseline. Second, the uint4 compact producer can write U8 x/w output and keep that output resident for an adjacent compact consumer; that producer-plus-consumer path measured 0.0458 ms, or 325.0x.

The kernel-only L2 result is the fastest measured row on this system, but it depends on compact input reuse and a working set that fits the persisting-L2 budget. On B200-class systems, very high HBM3e bandwidth narrows the cache advantage, so this tactic should be remeasured rather than assumed.

The L2 path does not make the workload compute-bound. It reduces read pressure, but the compact consumer still shows a memory-path profile because it writes the score stream and moves predictable global-memory transactions.

COMPACT OUTPUT FOOTPRINT **32 MiB**	MEASURED L2 CACHE **126 MiB**	PERSISTING-L2 BUDGET **79 MiB**
KERNEL-ONLY L2 RESULT **0.0233 ms**	L2 LIFT ON BEST PRODUCER **1.01x**	UINT4 PRODUCER LIFT **1.18x**
SCALAR L2 VS THRASH **1.55x**	WARM VS COLD CONSUMER **1.3x**	PERSISTING TOTAL LIFT **1.13x**
UINT4 PIPELINE LIFT **1.36x**	FUSED SCORE RESULT **0.0250 ms**	FUSED VS BEST ADJACENT **1.83x**
FUSED VS STRICT BASELINE **595.9x**		

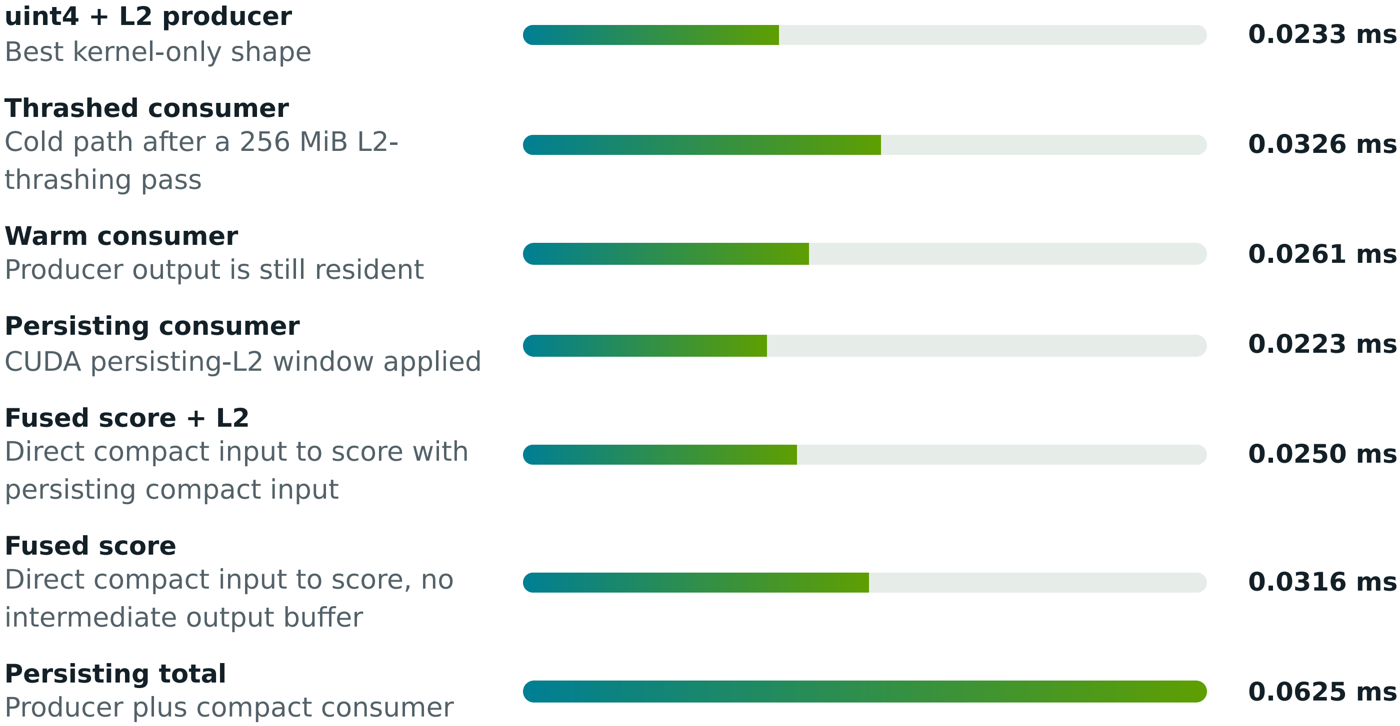
Measured latency path

Producer after L2 thrash
Current best kernel shape after cache pressure — **0.0427 ms**

Producer warm
Current best kernel shape with natural reuse — **0.0319 ms**

Producer with persisting input
Current best kernel shape plus L2 input residency — **0.0276 ms**

uint4 producer
Eight compact groups per thread — **0.0236 ms**



Cache residency planning model

Planning estimate only: assume about 70% of advertised cache is usable for resident working data, then verify on the target SKU. Aggregate cache helps only when the workload is partitioned so each GPU or die keeps its shard local.

GPU FAMILY	USABLE CACHE ESTIMATE	256 MIB WORKING SET	512 MIB WORKING SET	READOUT
H100 / H200	~35 MiB	8 GPUs	15+ GPUs	L2 density is too small for this tactic at practical scale.
RTX 6000 Ada / RTX 5090	~67 MiB	4 GPUs	8 GPUs	Good single-die cache, but still needs sharding for larger working sets.
RTX Pro 6000 Blackwell	~90 MiB	3 GPUs	6 GPUs	Best fit among workstation-class NVIDIA parts in this planning model.
B200	~180 MiB logical	2 GPUs	3 GPUs	High HBM3e bandwidth narrows the cache advantage; use L2 mainly for reuse and latency.
B300 SXM6 AC (measured)	~89 MiB	3 GPUs	6 GPUs	Measured in this B300 run: 126.5 MiB L2 and 79.1 MiB persisting budget.

Hardware Key Result Comparison

Representative rows from earlier L4 and RTX PRO 6000 runs are shown alongside the current measured system. The RTX PRO 6000 two-GPU entry is a topology note only; no multi-GPU timing is claimed.

HARDWARE	REPRESENTATIVE OPTION	ROLE	RESULT	READOUT
NVIDIA L4	Vectorized float default	Safe drop-in	2.31 ms / 15.1x	Earlier L4 campaign measured the float-compatible optimization track; compact and L2 variants were not part of that run.
RTX PRO 6000 Blackwell, 1 GPU	Vectorized float default	Safe drop-in	0.310 ms / 42.2x	Same float input/output contract as the baseline benchmark.
RTX PRO 6000 Blackwell, 1 GPU	uint4 + L2 U8 in/out kernel	Specialized kernel-only	0.0169 ms / 774.9x	This is the maximum-performance kernel-only option: compact U8 input/output with persisting input L2. It is not a drop-in float path and does not include downstream score work.
RTX PRO 6000 Blackwell, 1 GPU	uint4 + L2 compact pipeline	Specialized producer + consumer	0.0419 ms / 312.1x	Adjacent compact producer and compact consumer timed together; useful for coupled custom-data pipelines.
RTX PRO 6000 Blackwell, 2-GPU host	Topology note	Hardware context	PHB path, no NVLink timing claim	The earlier two-GPU host showed PCIe/PHB connectivity. No multi-GPU timing was committed, so this is not a performance row.
Current run: NVIDIA B300 SXM6 AC	Vectorized float default	Safe drop-in	0.185 ms / 80.6x	Recommended production default when the float input/output contract must remain unchanged.
Current run: NVIDIA B300 SXM6 AC	uint4 + L2 U8 in/out kernel	Specialized kernel-only	0.0233 ms / 638.7x	Fastest kernel-only row in the current measurement set; it requires the compact data contract.
Current run: NVIDIA B300 SXM6 AC	Fused compact U8 score + L2	Specialized score path	0.0250 ms / 595.9x	Best score-producing row in the current report; it is separate from the kernel-only producer result.

Multi-GPU Outlook

Single-GPU host

This run does not contain a multi-GPU opportunity because only one GPU is visible.

The current 8192 x 8192 harness fits on one GPU with headroom, so the next multi-GPU benchmark should be a row-sharded throughput test, not a replacement for the single-GPU score above.

ONE-GPU HARNESS MEMORY	PARTITIONED HARNESS FITS
784.00 MiB	**yes**

GPU-TO-GPU PATH	NVLINK
unknown	**no**

Run Context

ITEM	VALUE
GPU	NVIDIA B300 SXM6 AC
GPU count	1
GPU interconnect	unknown (no NVLink)
Driver	580.126.09
CUDA compiler	Cuda compilation tools, release 13.0, V13.0.88
Problem size	8192 x 8192

Source Package Links

These relative links are intended for the source bundle sent to the client, so the report, README, and implementation stay connected when reviewed offline.

SOURCE	PURPOSE
README	Project overview, hardware scoreboards, and experiment ledger.
CUDA source	Optimized kernel, affine/compact variants, and L2 experiment harness.
Report renderer	HTML report generator used for this PDF and published page.